

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра №22 «Кибернетика»

Направление подготовки 09.03.04 Программная инженерия

Отчёт

по летней практике студента на тему:

Реинжиниринг и развитие веб-платформы учёта и присваивания достижений

Группа Б23-504

Студент

(подпись)

Бученков В. М.

Руководитель

(подпись)

Рословцев В. В.

Научный консультант

(подпись)

(ФИО)

Оценка руководителя _____

Оценка консультанта _____

Итоговая оценка _____

ECTS _____

Комиссия Председатель

Рословцев В. В.

Самсонович А. В.

Тихомирова А. Н.

Шапкин П. А.

Зайцев В. С.

Москва 2026



Институт интеллектуальных кибернетических систем

КАФЕДРА КИБЕРНЕТИКИ

Задание на летнюю практику

Студенту гр. Б23-504 Бученкову Владиславу Максимовичу

(группа)

(ФИО)

ТЕМА ПРАКТИКИ

Реинжиниринг и развитие веб-платформы учёта и присваивания достижений

ЗАДАНИЕ

№ п/п	Содержание работы	Форма отчётности	Срок исполнения	Отметка о выполнении
1.	Изучение и анализ понятия реинжиниринга программного обеспечения и методов реинжиниринга, применяемых при развитии существующих веб-приложений	Аналитический обзор, разделы отчёта		
2.	Инспекция архитектуры существующей платформы и разработка решений по реинжинирингу её модульной структуры и клиент-серверного взаимодействия	Описание архитектуры, разделы отчёта		
3.	Реинжиниринг слоя доступа к данным с разделением контуров LMS и подсистемы достижений, выделением заменяемых интеграционных контрактов и подготовкой воспроизводимого локального развёртывания	Структура модулей, исходный код, разделы отчёта		
4.	Доработка механизмов идентификации пользователей, управления сессиями, защиты изменяющих запросов, ролевого и объектного разграничения доступа	Исходный код, описание механизмов защиты, разделы отчёта		
5.	Развитие кабинета преподавателя и администратора, включая управление назначениями преподавателей, достижениями и критериями их выдачи, точечный отзыв выданных достижений и ведение журнала аудита	Исходный код, пользовательский интерфейс, разделы отчёта		

№ п/п	Содержание работы	Форма отчётности	Срок исполнения	Отметка о выполнении
6.	Апробация результатов реинжиниринга на основных пользовательских и сквозных сценариях, проверка разграничения доступа, работы базы данных и корректности взаимодействия компонентов платформы	Сценарии и результаты проверки, разделы отчёта		
7.	Оформление отчёта, дневника практики и иллюстративного материала для доклада	Текст ПЗ, дневник, презентация		

ЛИТЕРАТУРА

1.	Roslovtssev V. V. Architecture for Modular Type System for Information Systems Based on Relational-Applicative Technologies // Procedia Computer Science. 2018. Vol. 123. Elsevier. P. 393–402.
2.	Рословцев В. В. Концептуализация предметных областей в аппликативной вычислительной среде // ScientificWorldJournal. 2017. Вып. 15, т. 1. Минск: ЧП «Ёлнать». С. 73–85.
3.	Рословцев В. В., Кононов В. М., Николайчик А. Е., Эргашев О. И. Реализация программного каркаса для реляционно-аппликативной вычислительной среды // ScientificWorldJournal. 2017. Вып. 15, т. 1. Минск: ЧП «Ёлнать». С. 85–93.
4.	Дейт К. Дж. Введение в системы баз данных. 8-е изд.: пер. с англ. М.: Издательский дом «Вильямс», 2005. 1328 с.: ил.
5.	Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. 4-е изд. СПб.: Питер, 2013. 896 с.
6.	Дейт К. Дж., Дарвен Х. Основы будущих систем баз данных. Третий манифест. 2-е изд.: пер. с англ. С. Д. Кузнецова, Т. А. Кузнецовой; под ред. С. Д. Кузнецова. М.: Янус-К, 2004. 656 с.
7.	Abramsky S., Jung A. Domain Theory. Corrected and expanded version [Электронный ресурс]. URL: https://www.cs.bham.ac.uk/~axj/pub/papers/handy1.pdf .

Дата выдачи задания: _____ Руководитель _____

Рословцев В. В.

«__» _____ 2026 г. Студент _____

Бученков В. М.

Реферат

Пояснительная записка содержит 22 страницы основного текста, 28 страниц с учётом иллюстративного и справочного материала, 7 рисунков, 9 таблиц и 17 использованных источников.

Ключевые слова: РЕИНЖИНИРИНГ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ, ВЕБ-ПЛАТФОРМА, ПОДСИСТЕМА ДОСТИЖЕНИЙ, LMS, МОДУЛЬНАЯ АРХИТЕКТУРА, BLAZOR SERVER, ASP.NET CORE, ENTITY FRAMEWORK CORE, POSTGRESQL, АУТЕНТИФИКАЦИЯ, АВТОРИЗАЦИЯ, АУДИТ.

Объектом исследования является веб-платформа учёта и присваивания достижений студентов.

Предмет исследования составляют методы и программные решения реинжиниринга архитектуры, разделения предметных областей, управления сессиями и разграничения доступа.

Цель работы заключается в проведении реинжиниринга существующей платформы и её развитии для поддержки студента, преподавателя и администратора.

В работе рассмотрены понятие и методы реинжиниринга программного обеспечения, проведена инспекция исходной архитектуры и построена целевая структура платформы. Определены границы LMS и подсистемы достижений. Реализованы разделённые контексты данных, интеграционные интерфейсы, защищённые сессии и объектные права. Приведены методика и результаты апробации.

Результатом стала модульно организованная версия платформы с отдельным контуром достижений, временным адаптером LMS, универсальным входом по GUID, серверными сессиями и CSRF-защитой. Созданы кабинеты сотрудников, управление назначениями, достижениями и критериями, точечный отзыв выдач и постоянный аудит событий.

Содержание

Введение	4
1 Анализ реинжиниринга программного обеспечения и исходного состояния платформы	6
1.1 Понятие реинжиниринга программного обеспечения	6
1.2 Методы реинжиниринга и область их применения	7
1.3 Инспекция исходной архитектуры платформы	8
1.4 Постановка задачи реинжиниринга	9
1.5 Выводы	10
2 Требования и целевая архитектура веб-платформы	11
2.1 Требования к результату реинжиниринга	11
2.2 Компонентная структура целевой системы	12
2.3 Граница LMS и подсистемы достижений	12
2.4 Идентичность, сессии и разграничение доступа	13
2.5 Выводы	14
3 Программная реализация реинжиниринга и развития платформы	16
3.1 Разделение контекстов данных и локальная инфраструктура	16
3.2 Идентификация пользователей и управление сессиями	16
3.3 Кабинет сотрудников и управление достижениями	17
3.4 Аудит и взаимодействие Blazor Server	18
3.5 Выводы	20
4 Апробация результатов реинжиниринга	21
4.1 Методика проверки	21
4.2 Результаты автоматизированной и сквозной проверки	21
4.3 Оценка результатов и оставшиеся ограничения	23
4.4 Выводы	24
Заключение	25
Список литературы	26

Введение

Цифровая образовательная среда развивается как совокупность связанных сервисов. Электронные ведомости, каталоги дисциплин и панели прогресса используют общие сведения о студентах и курсах, жизненный цикл которых определяется разными подсистемами. На раннем этапе проекта компоненты могут работать с единой базой и общими сущностями. По мере развития такая структура усложняет изменения, так как новое полномочие затрагивает несколько слоёв приложения и повышает риск нарушения уже проверенных сценариев.

Платформа учёта и присваивания достижений создаётся как часть образовательного веб-приложения кафедры кибернетики НИЯУ МИФИ. Достижения отражают освоенные темы, выполненные лабораторные работы и другие значимые результаты. Исследования геймификации связывают подобные механики с вовлечённостью и наглядным восприятием учебной траектории [1]. Прогресс в платформе представлен графом, вершины которого соответствуют достижениям, а рёбра задают зависимости. Его визуальная структура выражает отношения предметной области и имеет общие черты с графическими предметно-ориентированными языками [2].

На предыдущем этапе были разработаны модели достижений и электронных ведомостей, цикл присваивания, студенческий REST API, хранение в PostgreSQL и XML-граф для компонента визуализации. Практическая проверка выявила вопросы дальнейшего развития. Сведения о людях, курсах и зачислениях относились к будущей LMS, хотя обрабатывались общим контекстом данных. Вход поддерживал студенческую роль. Кабинеты сотрудников отсутствовали, а часть Blazог-интерфейса обращалась к собственному серверу через JavaScript и HTTP-контроллеры.

Летняя работа посвящена реинжинирингу существующей системы с сохранением её назначения и накопленных результатов. Потребовалось выделить границы LMS и подсистемы достижений, усилить защиту пользовательских операций и реализовать функции преподавателя и администратора. Актуальность этой задачи определяется потребностью образовательных веб-систем в сопровождаемой архитектуре. Они обрабатывают персональные и учебные данные, объединяют пользователей с разными полномочиями и зависят от внешних источников. Ошибки в границах ответственности приводят к дублированию данных и усложняют интеграцию, а ошибки авторизации создают угрозу конфиденциальности и целостности.

Объектом работы является веб-платформа учёта и присваивания достижений студентов. Предмет исследования включает методы реинжиниринга её архитектуры, модели разграничения данных и доступа, а также программные средства развития кабинета преподавателя и ад-

министратора.

Цель работы заключается в проведении реинжиниринга существующей платформы и развитии её архитектуры, механизмов защиты и пользовательских средств для поддержки студента, преподавателя и администратора. Для достижения цели изучены понятие и методы реинжиниринга, выполнена инспекция исходной архитектуры, разделены контуры LMS и достижений. Разработаны серверные сессии, ролевая и объектная авторизация. Реализованы кабинет сотрудников, управление назначениями, достижениями и критериями, отзыв выдач и аудит событий. Полученная версия проверена модульными, интеграционными и сквозными сценариями.

Практическая значимость состоит в подготовке платформы к подключению LMS и расширению преподавательского контура. Контракт учебного источника допускает замену локальной реализации внешним адаптером. Серверная проверка полномочий учитывает роль и конкретный экземпляр курса, а аудиторская история сохраняет существенные сведения о выдаче и отзыве достижения.

Первый раздел содержит анализ реинжиниринга и исходного состояния платформы. Во втором разделе сформулированы требования и представлена целевая архитектура. В третьем описана программная реализация. Четвёртый раздел посвящён апробации и ограничениям выполненного этапа.

1. Анализ реинжиниринга программного обеспечения и исходного состояния платформы

1.1 Понятие реинжиниринга программного обеспечения

Программная система существует дольше отдельной версии исходного кода. В ходе эксплуатации вокруг неё формируются устойчивые сценарии, данные, соглашения об интерфейсах и ожидания пользователей. Возникает накопленный контекст, который нельзя восстановить простым переносом функций в новый проект. Поэтому развитие работающего приложения требует исследования уже сложившейся структуры. В проектировании информационных систем этот процесс начинается с анализа предметной области, состава компонентов и связей между ними [15].

Реинжиниринг программного обеспечения представляет собой целенаправленное исследование и преобразование существующей программной системы. Его результатом становится более понятная и управляемая структура, способная поддерживать актуальные требования. Прикладное назначение системы сохраняется. Пользователь продолжает решать знакомые задачи, а внутренняя организация получает новые границы, модели и механизмы взаимодействия. Такой характер работы отличает реинжиниринг от первоначальной разработки, где архитектура формируется до накопления эксплуатационных ограничений.

Исторически потребность в реинжиниринге связана с долговечностью программных систем. Корпоративные приложения сохранялись десятилетиями, поскольку ценность данных и отлаженных процессов превышала выгоду от полной замены. Современные веб-системы развиваются быстрее, сохраняя ту же закономерность. Практически полезное решение содержит проверенные правила и постепенно накапливает архитектурные затруднения. Последовательное преобразование позволяет сохранять эти правила и сопоставлять каждый шаг с известным поведением.

Термин объединяет несколько уровней работы. На уровне понимания выполняется обратный анализ. Исследователь восстанавливает фактическую архитектуру по коду, схеме данных, конфигурации и наблюдаемому поведению. На уровне улучшения применяется реструктуризация. Изменяется форма модулей, зависимостей или таблиц, при этом значимые функции сохраняются. На уровне развития выполняется прямое проектирование целевого состояния и внедрение новых возможностей. В реальном проекте эти уровни образуют один цикл, представленный на рисунке 1.1.

Цикл опирается на документацию, код, схему данных и автоматические тесты. Сопостав-

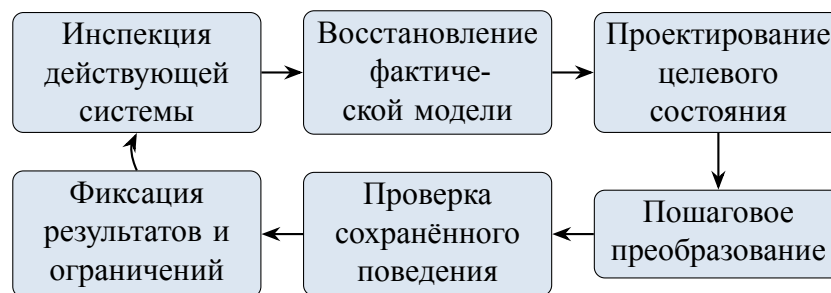


Рисунок 1.1 – Цикл реинжиниринга существующей программной системы

ление этих источников раскрывает фактические зависимости и устойчивое поведение. Затем предметные понятия отделяются от временных технических средств. Для платформы достижений устойчивыми являются достижение, критерий, связь и факт выдачи, а локальный каталог студентов выражает текущий способ интеграции. Преобразование завершается проверкой доменной логики, прикладных правил, HTTP-интерфейсов и работы с СУБД. Решения и оставшиеся ограничения фиксируются в проектных материалах, что делает процедуру воспроизводимой.

1.2 Методы реинжиниринга и область их применения

Методы реинжиниринга различаются глубиной воздействия и выбираются по причине затруднения. Локальная декомпозиция улучшает устройство класса, архитектурное выделение границы меняет распределение ответственности между подсистемами. Использованный набор приведён в таблице 1.1. Инспекция восстанавливает состояние системы, декомпозиция и инкапсуляция закрепляют владение, контракты ограничивают влияние внешней LMS, а регрессионная проверка подтверждает сохранение поведения.

Рефакторинг обеспечил перенос логики сессии в прикладной сервис, удаление общей JavaScript-обёртки и уточнение зависимостей проектов. Архитектурная модификация разделила общий EF Core контекст. Связи между LMS и достижениями выражены идентификаторами и прикладной проверкой через интерфейс, поэтому локальное устройство внешнего источника больше не закрепляется в доменной модели.

Проект пока не содержит production-базу с накопленными данными. Для воспроизводимой разработки выбран SQL-bootstrap чистой базы, а первая миграция контекста достижений отложена до стабилизации схемы. Внешняя совместимость оценивается по URI, форматам, авторизации и семантике операций. Контроллеры, Swagger и smoke-сценарий сохраняют REST-контракт [3, 4], в то время как серверный интерфейс вызывает application-сервисы напрямую.

Веб-приложение объединяет короткие HTTP-запросы, длительную сессию, интерактивное состояние и постоянные данные. Изменение механизма входа поэтому затрагивает контрол-

Таблица 1.1 – Методы реинжиниринга, применённые в работе

Метод	Назначение	Применение в платформе
Архитектурная инспекция	Восстановление состава компонентов и направлений зависимостей	Анализ проектов решения, контекста EF Core, HTTP API, Blazor-компонентов и тестов
Декомпозиция	Выделение модулей с определённой областью ответственности	Разделение LMS, подсистемы достижений, прикладных сервисов и пользовательского интерфейса
Инкапсуляция данных	Закрепление таблиц и ограничений за предметным контуром	Создание самостоятельного контекста достижений и временного локального контекста LMS
Выделение контракта	Сокращение зависимости от реализации внешнего источника	Введение интерфейсов чтения LMS и управления преподавательскими назначениями
Реструктуризация взаимодействия	Упрощение пути выполнения пользовательской операции	Прямой вызов прикладных сервисов из Blazor Server вместо общей JavaScript API-обёртки
Регрессионная проверка	Подтверждение сохранности поведения после преобразования	Модульные, интеграционные, архитектурные и сквозные сценарии

леры, Blazor-компоненты, cookie, antiforgery-защиту и тесты. ASP.NET Core предоставляет общий конвейер аутентификации и авторизации [6, 5], а предметные ограничения проверяет приложение. Роль запрещает студенту staff API, действующее назначение ограничивает преподавателя конкретным экземпляром курса.

Blazor Server хранит состояние компонента на сервере. Прямой вызов локального application-сервиса сокращает путь операции и исключает дополнительную сериализацию. HTTP и JavaScript сохранены на внешних границах, включая Vue-компонент графа. Изменяющие запросы защищены antiforgery-токеном, который согласуется с principal и состоянием сессии.

Форматы обмена остались стабильными. JSON применяется в HTTP API и ведомостях в соответствии с RFC 8259 [10], XML передаёт самодостаточное представление графа компоненту визуализации [11].

1.3 Инспекция исходной архитектуры платформы

Исходная версия платформы уже поддерживала завершённый студенческий сценарий. Пользователь входил по GUID, получал профиль и список активных курсов, открывал XML-граф, запускал цикл обработки ведомости и видел обновлённые состояния достижений. Данные хранились средствами Entity Framework Core, а прикладная логика была покрыта модульными и интеграционными тестами. Это состояние стало базовой линией реинжиниринга.

Инспекция показала несколько зон архитектурного напряжения. Единый

PlatformDbContext владел таблицами достижений и временными таблицами студентов, курсов, экземпляров и зачислений. Доменные ссылки пересекали границу будущей LMS. Вход был специализирован для студента, хотя модель разрешений уже предполагала преподавателя. Роль администратора и объектные права отсутствовали. Интерактивные страницы использовали общую JavaScript-обёртку собственного API. Локальная настройка опиралась на набор миграций, содержащий обе предметные области.

Связь проблем и принятых направлений реинжиниринга показана в таблице 1.2. Каждое направление сформулировано через наблюдаемое свойство системы. Это позволило проверять результат без оценки внутренней структуры по косвенным признакам.

Таблица 1.2 – Результаты инспекции и направления преобразования

Исходное состояние	Риск развития	Направление реинжиниринга
Общий контекст LMS и достижений	Закрепление внешней предметной области в схеме платформы	Отдельные контексты и контракт источника LMS
Вход только для студента	Невозможность безопасно развивать кабинеты сотрудников	Универсальное разрешение идентичности и серверные роли
Ролевая проверка без учебного объекта	Доступ преподавателя к чужому экземпляру курса	Объектная проверка активного назначения
JavaScript-обёртка для внутренних Blazor-страниц	Дублирование пути запросов и обработки сессии	Прямые вызовы application-сервисов
Отсутствие постоянной истории выдач и отзывов	Потеря контекста после удаления активной записи	Независимый журнал аудиторских событий
Миграции общей временной схемы	Смещение жизненных циклов LMS и платформы	SQL-bootstrap для локальной базы и отложенная чистая миграция достижений

Обнаруженные вопросы не требовали изменения алгоритма присваивания. AchievementProcessingCycle продолжил получать типизированную ведомость через IAppraisalPayloadProvider, вычислять факты и разрешать зависимости достижений. Сохранение этого контура стало принципиальным ограничением. Работа была направлена на окружение алгоритма, данные владения, доступ пользователей и управление определениями достижений.

1.4 Постановка задачи реинжиниринга

Целевое состояние задано проверяемыми свойствами. Контекст достижений хранит только собственные таблицы, сведения LMS читаются через интерфейс, а замена локального источника не изменяет прикладной цикл. Роль назначается сервером, изменяющие запросы проходят CSRF-проверку, преподаватель работает только со своими экземплярами курсов,

администратор управляет назначениями.

Функциональная часть охватывает создание и изменение достижений и критериев, управление их активностью, безопасное удаление, просмотр и точечный отзыв выдач. Каждая выдача и отзыв формируют постоянное аудиторское событие. Полный CRUD курсов и групп, редактор зависимостей, версии графа и публикация отнесены к следующему этапу. Успешность оценивается по разделению контекстов, ожидаемым кодам доступа, результатам автоматических тестов и воспроизводимому созданию PostgreSQL-базы.

1.5 Выводы

1. Реинжиниринг существующей программной системы включает инспекцию, восстановление фактической модели, проектирование целевого состояния, поэтапное преобразование и регрессионную проверку. Такой цикл позволяет развивать приложение с сохранением накопленного поведения.
2. Для платформы достижений выбраны методы архитектурной декомпозиции, инкапсуляции данных, выделения интеграционных контрактов, реструктуризации клиент-серверного пути и многоуровневого тестирования.
3. Инспекция выявила смешение данных LMS и достижений, ограниченную модель входа, отсутствие объектных прав и избыточный внутренний HTTP-путь Blazог. В выполненный этап вошли архитектура, безопасность, кабинеты сотрудников и аудит; редактор зависимостей, версии графа и полный LMS CRUD отнесены к дальнейшему развитию.

2. Требования и целевая архитектура веб-платформы

2.1 Требования к результату реинжиниринга

Требования объединяют предметные и эксплуатационные свойства. Первая группа охватывает студенческий граф, правила выдачи, учебные объекты и полномочия сотрудников. Вторая включает заменяемость LMS, защиту сессии, воспроизводимый локальный запуск и автоматическую проверку. Ролевые сценарии связывают архитектуру с наблюдаемым результатом. Студент работает со своими данными, преподаватель управляет назначенным курсом, администратор видит все экземпляры и определяет назначения. Требования и способы их подтверждения приведены в таблице 2.1.

Таблица 2.1 – Требования к целевой версии платформы

Код	Требование	Способ подтверждения
F1	Универсальный вход пользователя по GUID с назначением роли на сервере	Сценарии входа студента, преподавателя и администратора
F2	Доступ преподавателя только к действующим назначениям на экземпляры курсов	Проверки разрешённого и чужого курса с результатами 200 и 403
F3	Управление назначениями преподавателей администратором	Создание, изменение периода и завершение назначения
F4	Управление достижениями и критериями в доступном курсе	Сценарии создания, изменения, включения, отключения и удаления
F5	Точечный отзыв выдачи и чтение истории событий	Проверка сохранности других выдач и наличия события аудита
Q1	Независимость подсистемы достижений от структуры хранения LMS	Архитектурные тесты состава контекста и отсутствие межконтурных внешних ключей
Q2	Защита изменяющих HTTP-запросов	Проверка antiforgery-токена и безопасной политики cookie
Q3	Воспроизводимость локальной среды	Создание схемы SQL-bootstrap и загрузка согласованных демонстрационных данных
Q4	Сохранение действующего студенческого сценария	Регрессионные тесты графа, refresh и повторной выдачи

Сопровождаемость поддерживается структурой проектов и фиксацией причин, проверок и отложенных вопросов. Контракт LMS строится из фактических сценариев: поиска человека, чтения курсов и зачислений, проверки назначений и операций их изменения. Ограниченный состав снижает связанность и допускает будущий транспортный адаптер.

2.2 Компонентная структура целевой системы

Целевая архитектура развивает слоистую организацию решения. Внешний слой содержит Blazor Server, Razor Pages, HTTP API и Vue-компонент графа. Прикладной слой реализует сценарии и проверки доступа. Доменное ядро отвечает за критерии, факты ведомости, зависимости и цикл выдачи. Инфраструктурный слой предоставляет хранение достижений и временный адаптер LMS. Направление зависимостей следует от внешних деталей к контрактам предметной логики.

Компонентная схема показана на рисунке 2.1. В ней выделены два входа в прикладную логику. HTTP-контроллеры сохраняют внешний программный контракт и используются сквозной проверкой. Компоненты Blazor Server вызывают те же application-сервисы непосредственно. Оба пути используют общие проверки роли и учебного объекта. Поэтому различие транспорта не создаёт различия в правилах доступа.

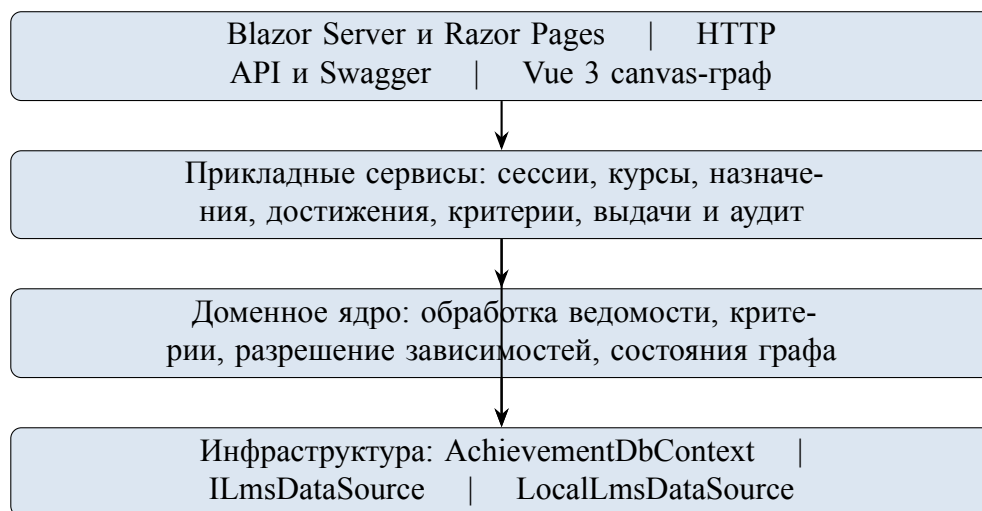


Рисунок 2.1 – Компонентная структура платформы после реинжиниринга

Прикладной сервис получает пользователя из claims, запрашивает учебные объекты, проверяет полномочие и обращается к данным достижений. Контроллер преобразует результат в HTTP-ответ, Blazor-компонент обновляет состояние страницы. Доменное ядро после проверки сценария извлекает факты ведомости, сопоставляет критерии и разрешает зависимости графа. Инфраструктурный слой отображает объекты на реляционную модель средствами Entity Framework Core [7, 8]. Контексты могут использовать одну локальную PostgreSQL-базу, сохраняя отдельные модели и правила владения.

2.3 Граница LMS и подсистемы достижений

Разделение данных стало центральным архитектурным решением. LMS владеет сведениями о людях, курсах, экземплярах, зачислениях, учебных группах и преподавательских на-

значениях. Подсистема достижений владеет определениями достижений, критериями, связями графа, активными выдачами и аудиторской историей. Между областями передаются GUID курса, человека и год экземпляра.

Граница представлена на рисунке 2.2. Прикладные сервисы не получают EF-сущности локальной LMS. Контракт возвращает минимальные DTO, которые описывают человека или экземпляр курса в объёме конкретного сценария. Временный LocalLmsDataSource преобразует локальные таблицы в эти DTO. Будущий HTTP-адаптер сможет реализовать тот же интерфейс и использовать ответ внешнего API.

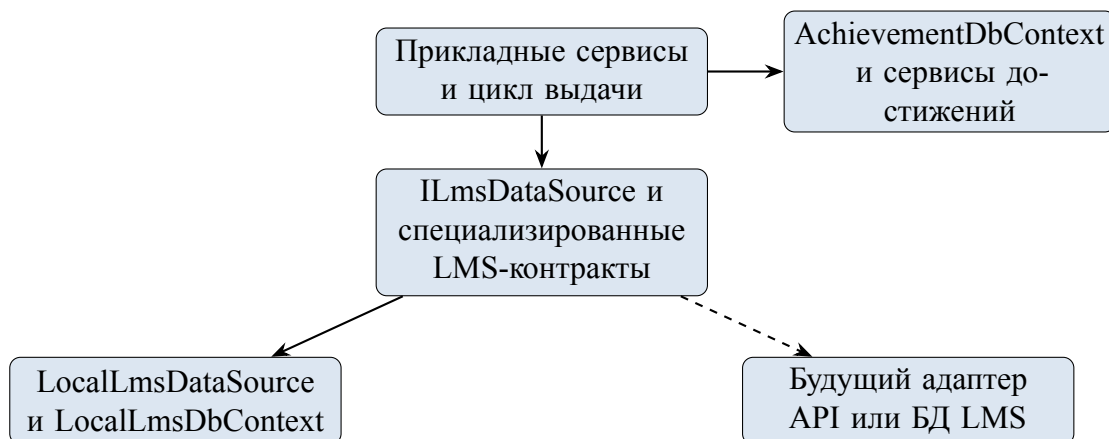


Рисунок 2.2 – Интеграционная граница между LMS и подсистемой достижений

Идентификаторы CourseID, Year и StudentID не имеют физических внешних ключей на LMS. Их корректность проверяется прикладным сервисом через контракт источника. Внутри контекста достижений критерии, выдачи и рёбра сохраняют реляционные связи, а уникальность пары студента и достижения предотвращает повторную активную выдачу [9, 13]. Интерфейсы LMS разделяют студенческое чтение, данные кабинета и изменение назначений. Реализация получает только необходимые ей полномочия.

2.4 Идентичность, сессии и разграничение доступа

Идентичность пользователя строится вокруг GUID, который уже используется в учебных данных. Пользователь вводит идентификатор на странице входа. Серверный resolver определяет существование субъекта и назначает роль. Студент находится через LMS. Разрешённые локальные преподаватели и администраторы на текущем этапе задаются конфигурацией. Клиент не передаёт желаемую роль, что исключает её самостоятельный выбор.

После успешного разрешения идентичности создаётся principal с claims. В них сохраняются идентификатор, отображаемое имя и роль. Authentication cookie содержит ключ серверного билета. Полные данные сессии находятся в InMemoryAuthenticationTicketStore. Выход удаляет билет и cookie. Изменение роли пользователя также приводит к недействительности

прежнего контекста после завершения сессии.

Последовательность входа и защищённого изменения показана на рисунке 2.3. Вход и выход реализованы обычными Razor Pages, поскольку установка cookie требует HTTP-ответа. Уже запущенный Blazor circuit использует сформированный principal. При обращении к изменяющему API браузер передаёт antiforgery cookie и соответствующий request token.

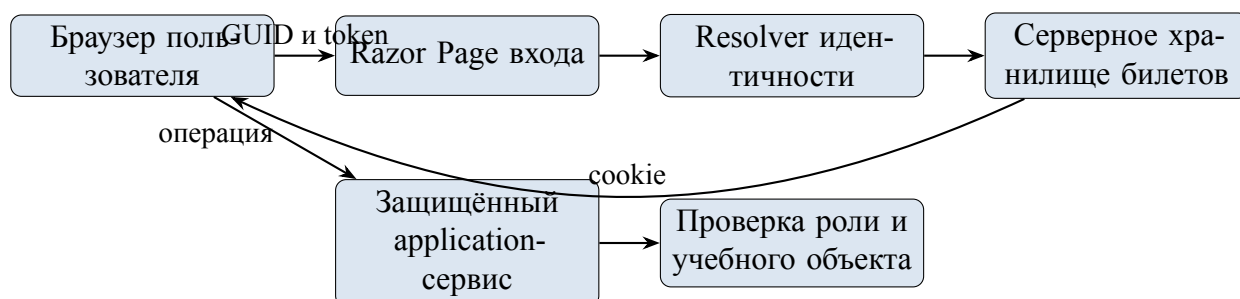


Рисунок 2.3 – Формирование сессии и выполнение защищённой операции

Cookie использует HttpOnly, SameSite=Lax и режим Secure в production. Срок жизни согласуется с серверным хранилищем билетов. Глобальный CSRF-фильтр защищает небезопасные методы. GET сохраняет семантику чтения, обновление графа выполняется POST, так как изменяет активные выдачи. Это согласуется с описанием API в OpenAPI [12].

Роль определяет общий класс действий. Студент использует собственные данные, преподаватель управляет достижениями доступного курса, администратор получает глобальные разрешения и управление назначениями. Объектная проверка использует активное назначение, заданное курсом, годом, GUID и периодом действия. Сервис проверяет его при открытии страницы и перед изменением. Матрица возможностей приведена в таблице 2.2. Признак ведущего преподавателя хранится в модели и пока не меняет состав операций.

Несуществующий экземпляр возвращает 404, чужой курс для преподавателя приводит к 403, запрос без сессии получает 401. Единый формат ошибки содержит стабильный код и сообщение. Эти правила действуют в Blazor Server и HTTP API. Скрытие ссылки помогает пользователю, окончательное решение принимает прикладной сервис после проверки роли, назначения и параметров.

2.5 Выводы

1. Требования к целевой версии связаны со сценариями трёх ролей, независимостью LMS, защитой изменяющих запросов и воспроизводимостью среды.
2. Слои пользовательского интерфейса, прикладных сервисов, доменного ядра и инфраструктуры получили определённые направления зависимостей. Blazor и HTTP API используют общие прикладные правила.

Таблица 2.2 – Матрица ролевых и объектных прав

Операция	Студент	Преподаватель	Администратор
Открыть собственный граф и запустить обработку	Да	Нет	Нет
Открыть кабинет курса	Нет	Только назначенный экземпляр	Любой существующий экземпляр
Управлять достижениями и критериями	Нет	Только назначенный экземпляр	Любой существующий экземпляр
Просматривать и отзываться выдачи	Нет	Только назначенный экземпляр	Любой существующий экземпляр
Читать аудит выдач и отзывов	Нет	Только назначенный экземпляр	Любой существующий экземпляр
Назначать преподавателей	Нет	Нет	Да

- Данные LMS и достижений разделены по владению и связаны GUID и узкими контрактами. Модель безопасности объединяет серверную роль, cookie-сессию, хранилище билетов, CSRF-защиту и проверку назначения на экземпляр курса.

3. Программная реализация реинжиниринга и развития платформы

3.1 Разделение контекстов данных и локальная инфраструктура

Общий контекст Entity Framework Core разделён на контекст достижений и временный локальный контекст LMS. Изменение затронуло регистрацию зависимостей, прикладные сервисы, тестовую инфраструктуру и подготовку базы. AchievementDbContext содержит определения, критерии, связи, активные выдачи и аудиторские события. LocalLmsDbContext хранит людей, курсы, зачисления, группы и назначения, не претендуя на роль окончательной модели внешней LMS. Распределение владения приведено в таблице 3.1.

Таблица 3.1 – Владение данными после разделения контекстов

Данные	Владелец	Правило связи
Достижения, критерии и связи графа	Подсистема достижений	Полные внешние ключи внутри контекста
Активные выдачи студентам	Подсистема достижений	Ссылка на достижение, GUID студента без связи с LMS
История выдач и отзывов	Подсистема достижений	Самодостаточный снимок без внешних ключей на изменяемые объекты
Люди и учебные группы	LMS	Доступ через проекции и идентификаторы
Курсы, экземпляры и зачисления	LMS	Проверка существования и доступа через LMS-контракт
Назначения преподавателей	LMS	Изменение через специализированный источник назначений

Достижение хранит CourseID и Year, факт выдачи хранит StudentID. Навигации к сущностям локальной LMS удалены. Проект Platform.Lms содержит DTO и интерфейсы студенческого чтения, кабинета и назначений. Локальный адаптер реализует их через LocalLmsDbContext, а прикладной слой не использует внешние EF-навигации.

Вместо миграций общей схемы создан scripts/sql/00_create_local_schema.sql. Подготовка среды ожидает PostgreSQL, применяет bootstrap и загружает демонстрационные данные. Скрипт не перестраивает старый Docker volume, поэтому новая схема проверяется на отдельной одноразовой базе. Чистая migration для AchievementDbContext будет сформирована после стабилизации production-схемы и включит только таблицы подсистемы достижений.

3.2 Идентификация пользователей и управление сессиями

Маршрут POST /api/auth/login принимает GUID. Resolver находит студента через LMS или сопоставляет сотрудника с локальной конфигурацией, после чего сервер назначает имя и роль. Отдельная таблица аккаунтов и парольный вход не вводились. Контракт resolver позволяет

позднее подключить каталог персон.

После входа GUID, имя и роль записываются в claims. Билет хранится в InMemoryAuthenticationTicketStore, cookie содержит его ключ. Выход и перезапуск приложения завершают локальную сессию; production-развёртыванию потребуется распределённое хранилище. HTTP-контроллер и Razor Page используют общий сервис пользовательской сессии и стабильные коды ошибок.

Небезопасные MVC actions защищены CSRF-фильтром. Клиент получает token через GET /api/auth/csrf и отправляет его в X-CSRF-TOKEN вместе с cookie. Вход и выход оформлены Razor Pages, поскольку authentication cookie должна устанавливаться в заголовке обычного HTTP-ответа до запуска Blazor circuit.

3.3 Кабинет сотрудников и управление достижениями

Кабинет сотрудников строится вокруг экземпляра курса. Преподаватель видит активные назначения, администратор получает все экземпляры. Страница курса повторно проверяет объектный доступ. Назначение содержит курс, год, GUID, период и признак ведущего. Сервис проверяет существование объектов, даты и пересечение назначений ведущих. Завершение изменяет конечную дату, сохраняя историю полномочий. Основные сервисы и маршруты приведены в таблице 3.2; контроллеры передают principal и параметры, а application-сервисы выполняют предметные проверки.

Таблица 3.2 – Прикладные сервисы и программные маршруты кабинета

Компонент	Назначение	Основные маршруты
Staff course service	Список экземпляров и объектная проверка доступа	GET /api/staff/courses, GET /api/staff/courses/{id}/{year}
Teaching assignment service	Чтение, сохранение и завершение назначения	GET/PUT /api/admin/courses/.../teachers, POST .../end
Achievement management service	CRUD достижений и критериев	GET/POST/PUT/DELETE /api/staff/courses/.../achievements
Award management service	Просмотр получателей и точечный отзыв	GET .../awards, DELETE .../awards/{studentId}
Audit query service	Чтение последних событий курса с фильтрами	GET .../award-audit

В Blazor-интерфейсе ссылка на кабинет отображается для преподавателя и администратора. Состав карточек определяется ролью, однако окончательная проверка всегда выполняется сервисом. Ошибка чужого курса показывается как понятное уведомление. Изменяющие действия требуют отдельного подтверждения, которое реализовано состоянием компонента.

Путь выполнения операции представлен на рисунке 3.1. Компонент получает теку-

щий principal через AuthenticationStateProvider и вызывает сервис. Сервис проверяет общую permission-модель, затем обращается к LMS за назначением. После подтверждения выполняется действие в контексте достижений или локальной LMS. Транзакция сохраняет данные и связанное событие.

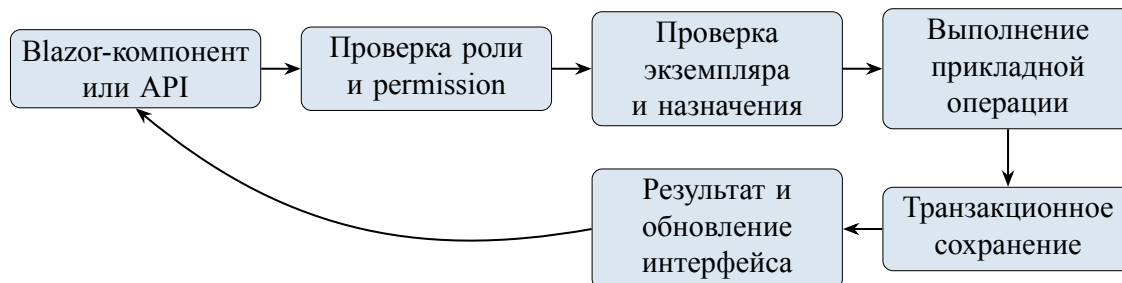


Рисунок 3.1 – Последовательность выполнения операции сотрудника

Страница достижений открывается в контексте выбранного курса и года. Сервис загружает только определения этого экземпляра. Преподаватель проходит объектную проверку. Администратор использует свой глобальный доступ. Студенческий principal отклоняется до чтения данных.

Определение достижения содержит название, описание, редкость, направление и идентификатор лабораторной работы. Сервис проверяет обязательные поля и уникальность названия, сохраняя идентификатор при изменении. Критерий задаёт теги, область SameMark, AcrossCourse или AllLabs и признак активности. Теги объединяются логическим AND. Интерфейс поддерживает создание, изменение, включение, отключение и подтверждаемое удаление критерия.

Определение со входящими или исходящими связями удалить нельзя, что защищает граф от висячих рёбер до появления редактора зависимостей. При допустимом удалении активные выдачи отзываются с причиной AchievementDeletion, затем удаляются критерии и определение. Снимки аудита сохраняются. Список получателей позволяет отозвать отдельную пару StudentID + AchievementID. Остальные выдачи остаются, а последующий POST refresh может повторить выдачу при выполненном критерии.

3.4 Аудит и взаимодействие Blazor Server

Таблица активных выдач описывает текущее состояние, постоянная история хранится в achievement_award_audit_events. Событие доступно только для добавления и чтения. Оно содержит тип, время, GUID студента, достижения и исполнителя, курс, год, роль, причину, исходную дату и предметные снимки. Структура остаётся читаемой после удаления определения или изменения LMS. Типы событий приведены в таблице 3.3. Активная запись и аудит сохраняются

одним SaveChanges, получая общую транзакционную судьбу.

Таблица 3.3 – События журнала выдач и отзывов

Тип	Причина	Исполнитель	Условие создания
Granted	CriteriaMatched	Система	Цикл обработки обнаружил выполненный критерий
Revoked	ManualRevocation	Преподаватель или администратор	Пользователь подтвердил точечный отзыв
Revoked	AchievementDeletion	Преподаватель или администратор	Определение удаляется вместе с активными выдачами

Поток формирования аудита показан на рисунке 3.2. Источником может быть автоматический цикл или ручная операция кабинета. Сервис формирует снимок до изменения активной строки. Затем контекст применяет обе записи в одной транзакции. Запрос журнала получает последние события курса и допускает фильтрацию по достижению или студенту.

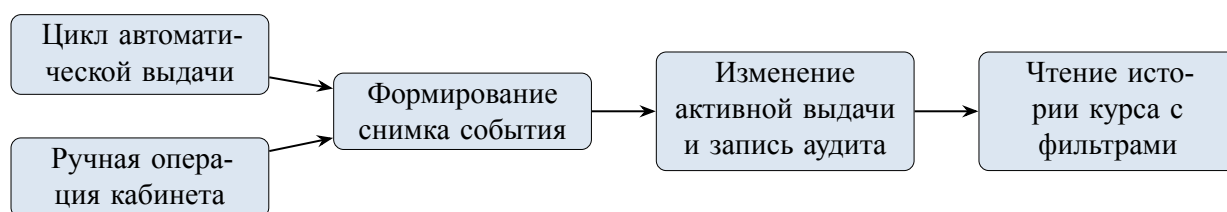


Рисунок 3.2 – Формирование и чтение аудиторской истории

Преподаватель читает журнал назначенного курса, администратор любого существующего курса, студент доступа не получает. Технические логи продолжают фиксировать запросы и исключения. Предметный аудит хранится в базе после перезапуска и пока охватывает выдачу и отзыв. История критериев, назначений и публикаций относится к дальнейшему развитию.

Исходные Blazor-страницы выполняли JavaScript interop, затем fetch к контроллеру и вызов application-сервиса. После перехода к Blazor Server этот внутренний сетевой контур стал избыточным. Общая обёртка удалена, компоненты внедряют сервисы и получают principal через AuthenticationStateProvider. Ролевые и объектные проверки сохранились.

HTTP-контроллеры продолжают обслуживать внешних клиентов, Swagger, тесты и smoke-сценарий. Внешний адаптер использует JSON и HTTP, серверный интерфейс вызывает C#-сервисы напрямую [14]. JavaScript применяется на границе Vue-компонента графа. Подтверждение опасных действий реализовано состоянием Blazor-компонента с явными кнопками подтверждения и отмены.

3.5 Выводы

1. Реализация разделила EF Core-модели достижений и временной LMS. Прикладной слой обращается к внешним учебным сведениям через минимальные интерфейсы и DTO.
2. Универсальный вход дополнен серверной ролью, хранилищем билетов и CSRF-защитой. Кабинеты сотрудников поддерживают объектные права, назначения, управление достижениями и критериями, просмотр получателей и точечный отзыв.
3. Аудит сохраняет выдачи и отзывы с предметным снимком в общей транзакции. Blazor Server вызывает application-сервисы напрямую, HTTP API остаётся внешним контрактом, JavaScript используется на границе визуализации графа.

4. Апробация результатов реинжиниринга

4.1 Методика проверки

Апробация подтверждает сохранение студенческого сценария и новые свойства архитектуры. Проверяются разделение LMS, серверные роли, объектные права и аудит выдач. Многоуровневая стратегия охватывает доменную логику, прикладные сервисы, HTTP-маршруты, метаданные EF Core, Vue-компонент и сквозной сценарий с PostgreSQL, cookie и antiforgery-токеном. Контуры на рисунке 4.1 позволяют локализовать ошибку на уровне правила, сервиса, API или реальной СУБД.

Доменный уровень подтверждает извлечение фактов, сравнение критериев и разрешение зависимостей. Прикладной уровень проверяет роли, учебные объекты и транзакции сервисов. Интеграционные сценарии запускают HTTP-маршруты на тестовом сервере, архитектурные проверки исследуют модель EF Core. Сквозной контур повторяет реальную последовательность пользователя и потому выполняется после быстрых наборов. Такое распределение ускоряет поиск причины отклонения и связывает каждое свойство системы с подходящим способом проверки.

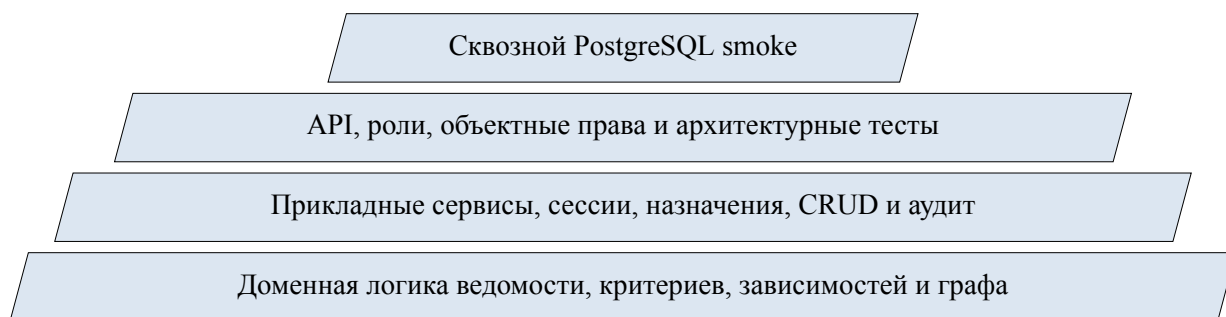


Рисунок 4.1 – Контуры автоматизированной апробации платформы

Демонстрационный студент зачислен на один курс, преподаватель назначен на первый из двух экземпляров, администратор видит оба. Набор различает отсутствие сессии, запрещённый и несуществующий объект. Изменяющие запросы после входа отправляют X-CSRF-TOKEN вместе с cookie jar, воспроизводя поведение браузера.

4.2 Результаты автоматизированной и сквозной проверки

Platform.Core.Tests проверяет ведомость, теги, критерии, зависимости, статусы графа и идемпотентность refresh. Platform.Application.Tests охватывает HTTP API, сессии, роли, объектный доступ, назначения, CRUD, отзыв, аудит и структуру EF-моделей. Vitest проверяет XML, состояния узлов и элементы отрисовки графа.

Набор приложения включает вход каждой роли, запрос без сессии, доступ преподавателя к назначенному курсу и запрет другого существующего экземпляра. Отдельные сценарии проверяют администратора, управление назначениями, достижениями и критериями, а также чтение аудита после отзыва. Архитектурные тесты фиксируют отсутствие сущностей LMS и межконтурных внешних ключей в AchievementDbContext. Тем самым регрессия обнаруживается по наблюдаемому результату и по структуре модели.

Результаты приведены в таблице 4.1. Все наборы завершились успешно, ошибок сборки нет. Известные nullable-предупреждения EF-сущностей не изменяют сценарии текущего этапа.

Таблица 4.1 – Результаты автоматизированной проверки

Контур	Выполнено	Результат	Назначение
Platform.Core.Tests	75 из 75	Успешно	Доменная логика ведомости и графа
Platform.Application.Tests	30 из 30	Успешно	API, роли, объектные права, CRUD и аудит
Vue 3 / Vitest	7 из 7	Успешно	Разбор и визуальная модель XML-графа
Сборка решения	0 ошибок	Успешно	Компиляция и согласованность проектов
git diff –check и bash -n	Без замечаний	Успешно	Формат изменений и синтаксис shell-скриптов

В ходе расширения тестов был обнаружен запрос, который успешно работал с InMemory-провайдером и не переводился Npgsql после проекции сущности в DTO. Сортировка была перенесена до проекции. После этого полный PostgreSQL-сценарий кабинета и назначений завершился успешно. Этот случай показывает ценность реальной СУБД в апробации. Провайдеры EF Core обладают разными возможностями перевода выражений, поэтому интеграционная проверка дополняет быстрые тесты.

Smoke-скрипт создаёт локальную базу, применяет bootstrap и seed, запускает приложение и выполняет HTTP-последовательность. Она включает CSRF-токен, входы трёх ролей, профиль, курсы, граф, refresh, чужой объект, назначения, CRUD, точечный отзыв и аудит. Ключевые сценарии в таблице 4.2 связывают архитектурные решения с наблюдаемым результатом.

Граница LMS подтверждается получением учебных данных через локальный адаптер. Различие двух существующих экземпляров курса проверяет объектную авторизацию точнее, чем запрос несуществующего идентификатора. Совместное изменение активной выдачи и появление события подтверждают транзакционность аудита. Отрицательные сценарии дополнительно различают ответы 401, 403 и 404.

После добавления аудита smoke получил 401, поскольку журнал запрашивался до

Таблица 4.2 – Основные сквозные сценарии платформы

Сценарий	Действие	Ожидаемый результат
Студенческий граф	Вход студента, чтение курса, GET и POST refresh	Граф изменяет доступное достижение на полученное, повторный refresh не создаёт дубликат
Объектный запрет	Преподаватель открывает второй существующий курс	Ответ 403 с кодом <code>course_access_denied</code>
Административное назначение	Администратор сохраняет период и ведущего преподавателя	Назначение доступно при повторном чтении и влияет на список курсов
Управление критерием	Сотрудник создаёт и отключает критерий в доступном курсе	Изменение сохраняется и учитывается циклом обработки
Точечный отзыв	Сотрудник отзываёт одну выдачу	Исчезает выбранная активная строка, остальные выдачи сохраняются
Аудиторская история	После выдачи и отзыва читается журнал курса	Присутствуют события <code>Granted</code> и <code>Revoked</code> с исполнителем и причиной

входа преподавателя. Проверка перенесена после `teacher login`, отзыва и повторной выдачи. Application-тесты подтверждают правила аудита, предыдущий PostgreSQL прогон проверил кабинет, права и назначения. Полный запуск исправленной последовательности на чистом disposable volume остаётся контрольной операцией.

4.3 Оценка результатов и оставшиеся ограничения

Метаданные EF Core подтверждают отсутствие LMS-сущностей и межконтурных внешних ключей. Ролевые сценарии запрещают студенту `staff API`, ограничивают преподавателя назначенным курсом и открывают администратору все экземпляры. CRUD-тесты подтверждают защиту связанных вершин, точечный отзыв, повторную выдачу и сохранение аудиторской записи.

Ограничения текущего этапа включают внешнюю LMS, production-миграции, распределённые сессии, редактор зависимостей, версии и публикацию графа. Обновление транзитивных Node.js-зависимостей требует отдельной регрессионной проверки визуализации. Выполненный блок поддерживает три роли, объектный доступ, управление достижениями и постоянную историю выдач, сохраняя основу для дальнейшего развития.

Предметный аудит пока охватывает выдачу и отзыв. Изменения критериев, назначений, зависимостей и публикаций представлены техническими логами или отнесены к следующим этапам. Расширение истории потребует единой модели причин, исполнителей и снимков изменяемых объектов. Эта задача может развиваться внутри выделенного контура достижений без нарушения границы LMS.

4.4 Выводы

1. Апробация организована на доменном, прикладном, интеграционном, архитектурном и сквозном уровнях. Каждый контур подтверждает определённый класс свойств реинжиниринга.
2. Успешно выполнены 75 тестов доменного ядра, 30 тестов приложения и 7 тестов Vue-компонента. PostgreSQL smoke подтвердил студенческий сценарий, кабинеты и назначения; финальный прогон исправленного порядка запросов на чистой базе остаётся контрольной операцией.
3. Архитектурные и ролевые цели достигнуты. Дальнейшие этапы включают внешнюю LMS, production-миграции, распределённые сессии, редактор зависимостей, версии графа и расширенный аудит.

Заключение

В ходе летней практики проведён реинжиниринг веб-платформы учёта и присваивания достижений. Выполненная работа развивает созданные ранее модели достижений и ведомостей, цикл присваивания, студенческий REST API и XML-представление графа.

Аналитическая часть включала изучение понятия и методов реинжиниринга, инспекцию кода, схем данных, интерфейсов и тестов. В результате были определены основные направления преобразования. К ним относятся декомпозиция контекстов, выделение интеграционных контрактов, реструктуризация клиент-серверного пути и многоуровневая регрессионная проверка.

Целевая архитектура закрепляет принадлежность сведений о людях, курсах, зачислениях и назначениях контуру LMS. Определения достижений, критерии, связи, активные выдачи и аудиторские события образуют самостоятельную предметную область. Связь между контурами выражена идентификаторами, минимальными DTO и интерфейсами. Такой подход допускает замену локального источника адаптером внешней LMS без перестройки доменного цикла.

Механизм входа расширен до трёх ролей. Сервер разрешает GUID, формирует claims и хранит билет сессии. Изменяющие HTTP-запросы проходят antiforgery-проверку, а полномочия преподавателя подтверждаются действующим назначением на экземпляр курса. Реализованы кабинеты преподавателя и администратора, управление назначениями, достижениями и критериями, точечный отзыв выдач и постоянный журнал аудита. Внутренние Blazor-страницы переведены на прямые вызовы application-сервисов, внешний HTTP API сохранён.

Корректность подтверждена 75 тестами доменного ядра, 30 тестами приложения и 7 тестами Vue-компонента. Проверены роли, объектный доступ, CRUD, аудит и архитектурные ограничения. Полный PostgreSQL smoke после последнего изменения порядка скриптов остаётся обязательной завершающей эксплуатационной проверкой.

Дальнейшее развитие связано с подключением настоящей LMS, подготовкой production-миграций, распределённым хранением сессий и редактором зависимостей графа. Сформированная архитектура создаёт для этих задач устойчивую основу.

Список литературы

1. From game design elements to gamefulness: defining gamification / S. Deterding [et al.] // Proceedings of the 15th International Academic MindTrek Conference. – New York : ACM, 2011. – P. 9–15. – DOI: 10.1145/2181037.2181040.
2. Литвинов Ю. В. Методы и средства разработки графических предметно-ориентированных языков : дис. ... канд. техн. наук : 05.13.11 / Ю. В. Литвинов ; С.-Петерб. гос. ун-т. – Санкт-Петербург, 2015. – 193 с.
3. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures : PhD dissertation. – Irvine : University of California, 2000. – 162 p. – URL: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm> (дата обращения: 20.02.2026).
4. Masse M. REST API Design Rulebook: Designing Consistent RESTful Web Service Interfaces. – Sebastopol : O'Reilly Media, 2011. – 116 p. – ISBN 978-1-4493-1050-9.
5. Freeman A. Pro ASP.NET Core 7. – 10th ed. – Berkeley : Apress, 2023. – 1308 p. – ISBN 978-1-4842-9333-0.
6. Introduction to ASP.NET Core [Электронный ресурс] // Microsoft Learn : официальная документация. – URL: <https://learn.microsoft.com/aspnet/core/introduction-to-aspnet-core> (дата обращения: 10.03.2026).
7. Smith J. P. Entity Framework Core in Action. – 2nd ed. – Shelter Island : Manning Publications, 2021. – 624 p. – ISBN 978-1-61729-836-3.
8. Entity Framework Core documentation [Электронный ресурс] // Microsoft Learn : официальная документация. – URL: <https://learn.microsoft.com/ef/core/> (дата обращения: 10.03.2026).
9. PostgreSQL 16 Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. – 2024. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 12.03.2026).
10. Bray T. The JavaScript Object Notation (JSON) Data Interchange Format : RFC 8259 [Электронный ресурс] / T. Bray ; Internet Engineering Task Force. – 2017. – URL: <https://www.rfc-editor.org/rfc/rfc8259> (дата обращения: 12.03.2026).
11. Extensible Markup Language (XML) 1.0 (Fifth Edition) : W3C Recommendation [Электронный ресурс] / T. Bray [et al.] ; World Wide Web Consortium. – 2008. – URL: <https://www.w3.org/TR/xml/> (дата обращения: 12.03.2026).
12. OpenAPI Specification v3.1.0 [Электронный ресурс] / OpenAPI Initiative. – 2021. – URL: <https://spec.openapis.org/oas/v3.1.0> (дата обращения: 12.03.2026).
13. Дейт К. Дж. Введение в системы баз данных : пер. с англ. – 8-е изд. – Москва : Издательский дом «Вильямс», 2005. – 1328 с.
14. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. – 4-е изд. – Санкт-Петербург : Питер, 2013. – 896 с. – ISBN 978-5-496-00433-6.
15. Роев К. В. Информационные технологии. Анализ и проектирование информационных систем : учебное пособие. – Санкт-Петербург : Лань, 2025. – 128 с.
16. Магомедов Ш. Г. Технологии разработки клиентских веб-приложений : учебное пособие. – Санкт-Петербург : Лань, 2024. – 144 с.
17. Модульное тестирование в .NET [Электронный ресурс] // Microsoft Learn : официальная документация. – URL: <https://learn.microsoft.com/dotnet/core/testing/> (дата обращения: 05.04.2026).